Multimodal AI Study Companion

Hackathon Track D Project Documentation

Multimodal AI Hackathon Track D Team

October 2026

Contents

1 Executive Summary **3**

2 Project Objectives **3**

3 System Architecture **3**
3.1 High-Level Components . . . 3
3.2 Request and Data Flow . . . 4
3.3 Repository Layout . . . 4

4 Multimodal Ingestion Pipeline **4**
4.1 PDF Extraction . . . 4
4.2 Image Extraction . . . 5
4.3 Video Transcription . . . 5
4.4 Chunking . . . 5

5 ChromaDB Indexing and Retrieval **5**
5.1 Embedding and Indexing . . . 5
5.2 Permission-Scoped Hybrid Retrieval . . . 6
5.3 Citation Handling . . . 6

6 Storage, Metadata, and Access Control **6**
6.1 Supabase Storage . . . 6
6.2 Database Model . . . 6

7 Deletion and Index Consistency **7**
7.1 Why Deletion Needs Coordination . . . 7
7.2 Implemented Deletion Flow . . . 7

8 Backend Application **7**

9 Frontend Application **8**
9.1 Study Workspace . . . 8
9.2 Viewers and Deep Links . . . 8
9.3 Assessment and Mastery . . . 8

10 Current Implementation Status **8**

11 Testing and Verification **9**

12 Known Limitations **9**

13 Recommended Evaluation Plan **10**

14 Recommended Next Steps **10**

15 Conclusion **11**

1 Executive Summary

The Multimodal AI Study Companion is an AI-powered learning workspace for lecture videos, textbooks, slides, and images. The system converts uploaded learning material into searchable, source-aware knowledge. Students can ask questions, receive answers grounded in their own course material, open the supporting page or video timestamp, and practice with adaptive quizzes.

The current implementation combines a Next.js frontend, a FastAPI backend, Supabase PostgreSQL and Storage, a Python ingestion pipeline, ChromaDB hybrid retrieval, and Groq/Gemini language model integrations. Authentication is performed using a JWT issued by the companion authentication server.

The central design principle is:

> *Store course knowledge in retrieval, and use the language model for grounded tutoring behavior.*

This approach avoids placing private course documents into model weights and allows uploaded files to be replaced or deleted.

2 Project Objectives

1. Ingest PDFs, scanned PDFs, images, and lecture videos.
2. Preserve source metadata such as page numbers and video timestamps.
3. Support permission-scoped retrieval for personal and community files.
4. Generate grounded answers with verifiable source citations.
5. Provide deep links from citations to a PDF page or video timestamp.
6. Generate course-material-based quizzes with explanations.
7. Track learner mastery using a client-side Bayesian Knowledge Tracing (BKT)-style model.
8. Support the complete file lifecycle, including removal of indexed content after a student deletes an uploaded file.

3 System Architecture

3.1 High-Level Components

Component	Responsibility
Next.js frontend	Three-pane study workspace, authentication flow, resource list, chat stream, quiz interface, PDF/video/image viewers, and mastery visualization.
FastAPI backend	Authentication, authorization, resource APIs, chat and quiz endpoints, signed URLs, background ingestion, and integration with retrieval and language model services.
Supabase PostgreSQL	Communities, memberships, resources, documents, and chat metadata.
Supabase Storage	Private and community file storage using signed URLs.

Python ingestion package	File extraction, OCR, transcription, chunking, embedding, indexing, and indexed-document deletion.
ChromaDB	Persistent vector collection containing source-aware document and transcript chunks.
BM25	Keyword retrieval over permission-filtered chunks.
Groq/Gemini	Grounded answer generation and quiz generation.
JWT authentication server	Google sign-in integration and signed identity claims used by the FastAPI backend.

3.2 Request and Data Flow

Listing 1: Upload and question-answering flow

```
Upload:
Browser -> FastAPI upload route
       -> Supabase Storage and metadata row
       -> background ingestion task
       -> extraction/transcription
       -> token-aware chunking
       -> embeddings and ChromaDB upsert

Question:
Browser -> authenticated FastAPI chat route
       -> database membership lookup
       -> permission-scoped vector and BM25 retrieval
       -> reciprocal-rank fusion
       -> Groq/Gemini prompt with source metadata
       -> citation validation
       -> SSE answer stream or JSON response
```

3.3 Repository Layout

Listing 2: Relevant repository directories

```
backend/ FastAPI application, routers, authentication, LLM and bridge
frontend/ Next.js application and study workspace
ingestion/ Extraction, transcription, chunking and ChromaDB indexing
db/ Prisma schema and database migrations
auth-server/ Authentication service
README.md Project overview and setup information
```

4 Multimodal Ingestion Pipeline

4.1 PDF Extraction

The PDF parser uses PyMuPDF. It first attempts native text extraction for each page. If a page has no selectable text, the page is rendered to a PNG image at high resolution and sent to the Gemini OCR helper. The output preserves:

- original file name;
- one-based page number;
- extracted text;

- a boolean indicating whether text was found.

This two-stage approach supports both normal text PDFs and scanned documents.

4.2 Image Extraction

PNG, JPG, and JPEG files are represented as one-page documents. The image bytes are passed to Gemini OCR with the corresponding MIME type. The resulting document uses the same page-level shape as PDF extraction, which allows both formats to share the document chunking and indexing path.

4.3 Video Transcription

The video transcriber uses `faster-whisper`. MP4, MKV, MOV, AVI, and WebM files are supported by the ingestion script. Each non-empty transcript segment stores:

```
{
    "start_time": 12.345,
    "end_time": 18.920,
    "text": "The explanation of the concept starts here."
}
```

The detected language, language probability, model size, and timestamped segments are written to a JSON transcript.

4.4 Chunking

Document and transcript chunking uses the `cl100k_base` tokenizer. The default chunk size is 400 tokens with 50 tokens of overlap.

For documents, chunks remain associated with their page number. For transcripts, chunks combine consecutive segments and preserve the first segment's start time and the last segment's end time. Oversized transcript segments are split into smaller token-sized pieces.

Metadata field	Meaning
`file_name`	Original source file name.
`page_number`	PDF or image page location; null for video chunks.
`start_time`	Video start time in seconds; null for document chunks.
`end_time`	Video end time in seconds; null for document chunks.
`chunk_index`	Chunk position within the indexed source.
`document_id`	Stable database/resource identifier.
`owner_id`	Uploading user's identity.
`access_scope`	Either `private` or `community`.
`community_id`	Community identifier when sharing is enabled.

5 ChromaDB Indexing and Retrieval

5.1 Embedding and Indexing

The ingestion package uses the `all-MiniLM-L6-v2` sentence-transformer to embed non-empty chunks. Embeddings are normalized and stored in the persistent Chroma collection `course_material_scoped`.

Each Chroma record has a stable identifier derived from the database document identifier:

```
chunk_id = f"{document_id}_{chunk_index}_{page_number}_{start_time}"
```

The `document_id` is also stored explicitly in metadata. The metadata field is the authoritative key for deleting all chunks belonging to a source.

5.2 Permission-Scoped Hybrid Retrieval

The retrieval process is:

1. Build a private-document filter for the authenticated user.
2. Query the database for communities the user belongs to.
3. Add authorized community documents to the access filter.
4. Run Chroma vector retrieval using the combined filter.
5. Run BM25 keyword retrieval only over the same filtered records.
6. Combine vector and keyword rankings using Reciprocal Rank Fusion.
7. Convert the selected records into backend source metadata.

This prevents unauthorized private or community chunks from entering either the semantic or keyword search path.

5.3 Citation Handling

The LLM prompt includes a required source tag for every retrieved chunk. The citation guardrail scans generated answers, removes unmatched citation tags, and returns validated citation objects. PDF citations contain page numbers; video citations contain timestamp ranges.

The frontend resolves citations to a source and opens the appropriate viewer. Video citations seek the Video.js player, while PDF citations navigate to the referenced page. The current implementation supports the viewer's highlight metadata contract, although automatic OCR/text bounding-box generation is not yet complete.

6 Storage, Metadata, and Access Control

6.1 Supabase Storage

The application uses separate buckets:

- `private-materials` for personal files;
- `community-materials` for community files.

Storage paths include an identity and a stable resource identifier. The backend returns short-lived signed URLs only after checking ownership or community membership.

6.2 Database Model

The database contains the following principal entities:

- **Community**: shared learning space and invite code;
- **CommunityMember**: user membership relation;
- **Resource**: community resource metadata;
- **Document**: personal or indexed resource metadata;
- **Chat**: chat-session metadata.

The current processing states are:

$$\texttt{uploaded} \rightarrow \texttt{processing} \rightarrow \texttt{completed}$$

Failures use `failed`. Deletion uses `deleting` followed by `deleted`.

7 Deletion and Index Consistency

7.1 Why Deletion Needs Coordination

Ingestion runs as a background task. Without coordination, the following race could occur:

1. A file starts OCR or transcription.
2. The student deletes the file.
3. The delete route removes existing Chroma chunks.
4. The old background task finishes and inserts the chunks again.

7.2 Implemented Deletion Flow

The current worktree contains the following deletion flow:

1. The backend verifies ownership or community membership.
2. The metadata record is marked `deleting`.
3. All Chroma records matching the stable `document_id` are removed.
4. The Supabase Storage object is removed.
5. The metadata record is marked `deleted`.
6. Deleted resources are excluded from normal resource listings.

The background ingestion bridge checks whether the source record is missing, `deleting`, or `deleted`. A final cancellation check occurs after extraction/chunking and immediately before Chroma insertion.

The API routes are:

```
DELETE /api/personal/resources/{resource_id}
DELETE /api/communities/{community_id}/resources/{resource_id}
```

This design preserves document isolation and avoids deleting chunks by non-unique file names.

8 Backend Application

The FastAPI application exposes:

- health check endpoint;
- synchronous grounded chat endpoint;
- server-sent event chat endpoint;
- quiz generation endpoint;

- personal resource upload, listing, signed URL, and deletion;
- community creation, joining, membership, resource upload, signed URL, and deletion;
- chat-session listing and deletion.

The backend authenticates requests using a bearer JWT. The service-role Supabase client is kept on the backend, while the frontend receives only short-lived signed resource URLs.

9 Frontend Application

9.1 Study Workspace

The frontend provides a three-pane workflow:

1. **Sources**: personal and community materials;
2. **Chat**: streaming grounded questions and answers;
3. **Viewer**: PDF, video, and image source viewer.

9.2 Viewers and Deep Links

The PDF viewer uses `react-pdf` and supports page navigation. The Video.js player supports programmatic seeking to a citation timestamp. Images are displayed directly from signed Storage URLs.

When a selected resource is deleted, the viewer context detects that the resource is no longer present and clears the open viewer target.

9.3 Assessment and Mastery

The quiz interface supports multiple-choice questions, answer feedback, explanations, source-citation text, and mastery updates. The client-side BKT helpers classify topic mastery and adapt quiz length.

The current implementation provides the user-interface and local learner model foundation. Formal simulated-student results and a backend-persisted learner profile remain future evaluation work.

10 Current Implementation Status

Feature	Status	Notes
Native PDF extraction	Implemented	PyMuPDF page extraction.
Scanned PDF OCR	Implemented	Gemini OCR fallback.
Image OCR	Implemented	PNG/JPG/JPEG support.
Video transcription	Implemented	Timestamped faster-whisper output.
Token-aware chunking	Implemented	Documents and transcripts.
ChromaDB indexing	Implemented	Persistent embeddings and metadata.
Hybrid retrieval	Implemented	Vector search, BM25, and RRF.
Permission filtering	Implemented	Private owner and authorized communities.
Supabase Storage	Implemented	Uploads and signed URLs.

Personal deletion	Implemented in current worktree	Chroma, Storage, and metadata cleanup.
Community deletion	Implemented in current worktree	Membership-protected cleanup.
Deletion race protection	Implemented in current worktree	Processing state and cancellation checks.
Citation validation	Partially implemented	Non-streaming validation is present.
Streaming citation events	Incomplete	Frontend/backend event behavior needs alignment.
Grounded quiz retrieval	Incomplete	Quiz route needs topic-based server retrieval.
BKT personalization	Prototype	Client-side state and adaptive length.
Formal evaluation report	Not yet implemented	Requires benchmark data and metrics.

11 Testing and Verification

The repository includes tests for:

- PDF extraction and OCR fallback;
- image parsing;
- document routing;
- document and transcript chunking;
- ingestion pipeline routing;
- transcript metadata preservation;
- access-control-related behavior;
- storage and retrieval helper behavior.

The changed Python files were syntax-checked with `python -m compileall`. Whitespace validation passed with `git diff -check`. In the development environment used for this documentation, `pytest` was not installed and the frontend `node_modules` directory was unavailable, so the complete test suite, frontend lint, and frontend production build could not be executed there.

Before deployment, the database migration `db/prisma/migrations/0004_resource_processing_status/m` must be applied to the Supabase database.

12 Known Limitations

1. Streaming chat currently emits token events and completion events, but citation-event delivery is not yet fully aligned with the frontend citation contract.
2. The quiz frontend can request questions without first performing topic-based permission-scoped retrieval.
3. Community discovery is partly remembered in browser local storage rather than being fully reconstructed from a backend membership list.

4. PDF and image OCR text is preserved, but exact visual text coordinates are not generated by the ingestion pipeline.

5. Retrieval failures should be distinguished more clearly from an empty retrieval result.

6. Standalone ingestion storage helpers and backend upload routes contain overlapping storage logic and should eventually be consolidated.

7. Formal measurements for grounding, assessment quality, personalization, latency, and deletion consistency are not yet reported.

13 Recommended Evaluation Plan

To evaluate the system rigorously, create a labeled test set with the following fields:

Listing 3: Suggested evaluation record

```
{
  "id": "video-001",
  "question": "What is the purpose of the learning rate?",
  "topic": "gradient descent",
  "answerable": true,
  "expected_sources": [
    {"file": "lecture.mp4", "start_time": "02:15"}
  ],
  "expected_concepts": ["step size", "parameter update"]
}
```

Recommended metrics are:

- Retrieval Recall@1, Recall@3, Recall@5, and MRR;
- citation precision and citation recall;
- grounded-answer faithfulness;
- unsupported-query refusal rate;
- quiz correctness, source support, topic relevance, and duplicate rate;
- learner accuracy and mastery change across simulated quiz runs;
- upload-to-index latency, query latency, and failure rate;
- deletion consistency across database, Storage, and ChromaDB.

Results should include a baseline, the improved system, examples of failures, and known limitations. A small, reproducible test set is preferable to an unverified accuracy claim.

14 Recommended Next Steps

1. Apply and verify the resource-processing database migration.

2. Add integration tests for upload, indexing, deletion, and re-query.

3. Complete server-side topic retrieval for grounded quizzes.

4. Align streaming citation events with the frontend contract.

5. Add explicit unsupported-query refusal tests.
6. Run simulated learner experiments and report mastery gains.
7. Add a backend endpoint for listing all user communities.
8. Consolidate duplicate Storage/upload helper logic.
9. Run frontend lint/build after installing the frontend dependencies.
10. Record a demo showing upload, citation navigation, adaptive practice, unsupported-query refusal, and deletion from the knowledge base.

15 Conclusion

The current project has a strong multimodal RAG foundation. It already supports extraction from documents, images, and videos; source-aware chunking; hybrid permission-scoped retrieval; Supabase Storage; citation navigation; and adaptive quiz interfaces. The newly added deletion lifecycle keeps the persistent knowledge index consistent when an uploaded resource is removed.

The highest-value remaining work is not model fine-tuning. It is completing grounded quiz retrieval, making streaming citations fully reliable, and producing a rigorous evaluation report with measurable grounding and learner improvement results.